Intelligent Customer Support System Using Agentic AI Approach

Dataset Link: https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets

Student Name:

Student ID:

Table of Contents

1. Application and Workflow Selection 3

1.1 Introduction and Motivation 3

1.2 Selected Workflow with Diagram 3

1.3 Objectives of the Intelligent System.....4

1.4 Impact of the Project 5

2. Selection of AI Technologies 5

2.1 Identification of Similar Systems 5

2.2 Innovative Approaches 5

2.3 AI Methods, Tools, and Technologies 6

3. Simulations and Development Setup 6

3.1 Simulation Environment and Parameters 6

3.2 Agent-wise Task Distribution and Dataset Usage 8

3.3 Data Encoding and Processing.....8

4. Use Case Explanation with Evidence.....9

4.1 Step-by-Step Workflow Execution.....9

4.2 Results and Insights.....19

5. Conclusions.....21

References 22

1. Application and Workflow Selection

1.1 Introduction and Motivation

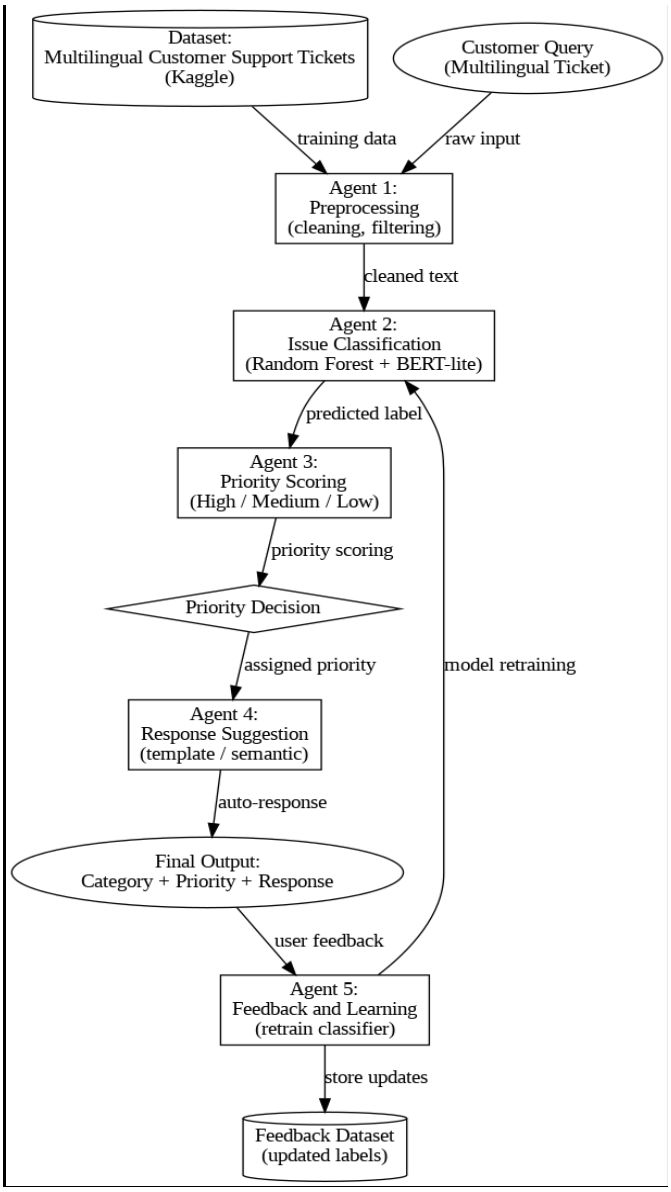
The speed of online service and digital communication development has resulted in a significant rise of the number of customer service queries. Organizations are now challenged to manage the multilingual and diverse queries with great efficiency and at the same time deliver high-quality services. Conventional manual support systems are usually characterized by slowness in responses and inconsistent levels of service and the overall cost of operation. In order to overcome these obstacles, this project will suggest the creation and deployment of a smart customer support system that utilizes Agentic AI, and several autonomous agents will cooperate to process, categorise and answer queries in contextual knowledge.

The background that led to the choice of customer support as the application area is due to the high demands of speed, accuracy, and personalization in dealing with clients. The search and resolution of high-priority or technical issues should be managed as quickly as possible so the system will reduce human labor, improve response time, classify issues by priority, and suggest responses as well as provide feedback to reduce human labor (Kowsari et al., 2019). Moreover, the concept of incorporation of multilingual support enables global organizations to offer standardized service in different regions and thus increase consumer satisfaction and effectiveness of the organizations.

1.2 Selected Workflow with Diagram

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Intelligent system- Kangathi.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
KAGGLE_USERNAME = "your_username_here"
KAGGLE_KEY = "your_api_key_here"

import json
import os

#  Create kaggle directory
os.makedirs('/root/.kaggle', exist_ok=True)

# Create kaggle.json with credentials
kaggle_api = {
    "username": KAGGLE_USERNAME,
    "key": KAGGLE_KEY
}

with open('/root/.kaggle/kaggle.json', 'w') as f:
    json.dump(kaggle_api, f)

# Set permissions
!chmod 600 /root/.kaggle/kaggle.json

# Install kaggle API if not already installed
!pip install -q kaggle

# Download the Multilingual Customer Support Tickets dataset
!kaggle datasets download -d tobiasbueck/multilingual-customer-support-tickets

# Unzip the downloaded dataset
!unzip -q multilingual-customer-support-tickets.zip
```

```
Dataset URL: https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
===== Random Forest Results =====
Accuracy: 0.6594202898550725
Macro F1: 0.14890173779708665

Classification Report:
                       precision    recall  f1-score   support

   Account Assistance       0.00      0.00      0.00         1
        Billing Issue       1.00      0.87      0.93        23
     Customer Service       0.00      0.00      0.00         2
      Feature Request       0.00      0.00      0.00         1
     Hardware Failure       0.00      0.00      0.00         1
           IT Support       0.67      0.17      0.27        24
          Login Issue       0.00      0.00      0.00         1
        Network Issue       1.00      0.14      0.25         7
          Order Issue       0.00      0.00      0.00         1
   Payment Processing       0.00      0.00      0.00         1
      Product Support       0.00      0.00      0.00        12
Returns and Exchanges       1.00      0.50      0.67        12
        Sales Inquiry       0.00      0.00      0.00         4
   Service Disruption       0.00      0.00      0.00         4
       Service Outage       0.00      0.00      0.00         7
         Software Bug       0.00      0.00      0.00         3
   Technical Guidance       0.00      0.00      0.00         1
    Technical Support       0.63      0.99      0.77       151
         Urgent Issue       0.50      0.05      0.10        19
       Warranty Claim       0.00      0.00      0.00         1

             accuracy                           0.66       276
            macro avg       0.24      0.14      0.15       276
         weighted avg       0.59      0.66      0.56       276
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Agent 1: Query Preprocessing Agent

```python
import os

# List all files in current directory
print(os.listdir())
```

```
['.config', 'dataset-tickets-multi-lang3-4k.csv', 'df_clean.csv', 'bert_encoder.pkl', 'multilingual-customer-support-tickets.zip', 'bert_classifier.pkl', 'tfidf.pkl', 'dataset-tickets-multi-lang-4-20k.csv', 'dataset-tickets-
```

```python
import pandas as pd

# Load dataset
df = pd.read_csv("dataset-tickets-multi-lang3-4k.csv")

# Check first few rows
print(df.head())

# Check column names
print(df.columns)
```

```
                                        subject  \
0  Problema crítico del servidor requiere atenció...
1     Anfrage zur Verfügbarkeit des Dell XPS 13 9310
2  Erro na Autocompletação de Código do IntelliJ ...
3            Urgent Assistance Required: AWS Service
4                Problème d'affichage de MacBook Air

                                                body  \
0  Es necesaria una investigación inmediata sobre...
1  Sehr geehrter Kundenservice,\n\nich hoffe, die...
2  Prezado Suporte ao Cliente <name>,\n\nEstou es...
3  Dear IT Services Support Team, \n\nI am reachi...
4  Cher équipe de support du magasin en ligne Tec...
```

```
                                        subject  \
0  Problema crítico del servidor requiere atenció...
1     Anfrage zur Verfügbarkeit des Dell XPS 13 9310
2  Erro na Autocompletação de Código do IntelliJ ...
3            Urgent Assistance Required: AWS Service
4                Problème d'affichage de MacBook Air

                                                body  \
0  Es necesaria una investigación inmediata sobre...
1  Sehr geehrter Kundenservice,\n\nich hoffe, die...
2  Prezado Suporte ao Cliente <name>,\n\nEstou es...
3  Dear IT Services Support Team, \n\nI am reachi...
4  Cher équipe de support du magasin en ligne Tec...

                                            answer      type  \
0  Estamos investigando urgentemente el problema ...  Incident
1  Sehr geehrter <name>,\n\nvielen Dank, dass Sie...   Request
2  Prezado <name>,\n\nObrigado por entrar em cont...  Incident
3  Dear <name>,\n\nThank you for reaching out reg...   Request
4  Cher <name>,\n\nMerci de nous avoir contactés ...  Incident

                queue priority language      business_type             tag_1  \
0  Technical Support     high       es        IT Services       Urgent Issue
1   Customer Service      low       de  Tech Online Store      Sales Inquiry
2  Technical Support     high       pt        IT Services  Technical Support
3         IT Support     high       en        IT Services         IT Support
4    Product Support      low       fr  Tech Online Store  Technical Support

                tag_2                tag_3             tag_4  \
0  Service Disruption       Incident Report  Service Recovery
1     Product Support      Customer Service       Order Issue
2        Software Bug    Problem Resolution      Urgent Issue
3        Urgent Issue  Service Notification    Cloud Services
4     Product Support      Hardware Failure  Service Recovery

                    tag_5               tag_6               tag_7 tag_8  tag_9
0     System Maintenance                 NaN                 NaN   NaN    NaN
1  Returns and Exchanges                 NaN                 NaN   NaN    NaN
2            IT Support                 NaN                 NaN   NaN    NaN
3     Problem Resolution  Technical Guidance  Performance Tuning   NaN    NaN
4        Routine Request                 NaN                 NaN   NaN    NaN
```

[redacted]

[redacted]

[redacted]

[redacted]

Agent 2 — Issue Classification Agent (Random Forest + BERT-lite)

```python
import pandas as pd
from sklearn.model_selection import train_test_split

# Load cleaned dataset
df = pd.read_csv("df_clean.csv")

# Keep only English
df = df[df['language'] == 'en']

# Filter classes with at least 2 samples
class_counts = df['tag_1'].value_counts()
valid_classes = class_counts[class_counts >= 2].index
df = df[df['tag_1'].isin(valid_classes)]

# Features and labels
X = df['text_clean']
y = df['tag_1']

# Split train-test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
print("Number of classes:", len(y_train.unique()))
```

```
Train size: 1104
Test size: 276
Number of classes: 23
```

```python
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, f1_score
import pickle

# Load cleaned dataset
df = pd.read_csv("df_clean.csv")

# Remove classes with less than 2 samples
class_counts = df['tag_1'].value_counts()
valid_classes = class_counts[class_counts >= 2].index
df = df[df['tag_1'].isin(valid_classes)]

# Features and labels
X = df['text_clean']
y = df['tag_1']

# Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Convert text to TF-IDF features
tfidf = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# Train Random Forest classifier
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train_tfidf, y_train)

# Predict on test set
y_pred = rf.predict(X_test_tfidf)
```

```python
# Evaluate
print("===== Random Forest Results =====")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Macro F1:", f1_score(y_test, y_pred, average='macro'))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Save model and TF-IDF vectorizer
with open("rf_model.pkl", "wb") as f:
    pickle.dump(rf, f)
with open("tfidf.pkl", "wb") as f:
    pickle.dump(tfidf, f)

print("\nSaved Random Forest model as rf_model.pkl")
print("Saved TF-IDF vectorizer as tfidf.pkl")
```

```
===== Random Forest Results =====
Accuracy: 0.6594202898550725
Macro F1: 0.14890173779708665

Classification Report:
                        precision    recall  f1-score   support

   Account Assistance       0.00      0.00      0.00         1
        Billing Issue       1.00      0.87      0.93        23
      Customer Service       0.00      0.00      0.00         2
       Feature Request       0.00      0.00      0.00         1
      Hardware Failure       0.00      0.00      0.00         1
            IT Support       0.67      0.17      0.27        24
           Login Issue       0.00      0.00      0.00         1
         Network Issue       1.00      0.14      0.25         7
           Order Issue       0.00      0.00      0.00         1
   Payment Processing        0.00      0.00      0.00         1
      Product Support        0.00      0.00      0.00        12
Returns and Exchanges       1.00      0.50      0.67        12
        Sales Inquiry       0.00      0.00      0.00         4
   Service Disruption       0.00      0.00      0.00         4
       Service Outage       0.00      0.00      0.00         7
```

```python
!pip install -q sentence-transformers

from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression

# Load BERT encoder
bert_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')

# Encode text
X_train_bert = bert_model.encode(X_train, show_progress_bar=True)
X_test_bert = bert_model.encode(X_test, show_progress_bar=True)

# Train simple classifier on embeddings
bert_clf = LogisticRegression(max_iter=1000, multi_class='multinomial')
bert_clf.fit(X_train_bert, y_train)

# Predictions
y_pred_bert = bert_clf.predict(X_test_bert)

# Evaluation
print("===== BERT-lite Classifier Results =====")
print("Accuracy:", accuracy_score(y_test, y_pred_bert))
print("Macro F1:", f1_score(y_test, y_pred_bert, average='macro'))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_bert))
```

```
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Batches: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 35/35 [01:26<00:00,  2.15it/s]
Batches: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 9/9 [00:21<00:00,  1.37s/it]
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it
  warnings.warn(
===== BERT-lite Classifier Results =====
Accuracy: 0.7065217391304348
Macro F1: 0.2219234255096843
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Agent 3: Multi-class Priority Scoring Agent

```python
import pandas as pd

# Load test set predictions
# Example with Random Forest:
df_test = pd.DataFrame({
    'text': X_test,
    'true_label': y_test,
    'pred_label': y_pred  # Random Forest predictions
})

# Define priority mapping for categories
priority_mapping = {
    'Urgent Issue': 'High',
    'Technical Support': 'High',
    'IT Support': 'Medium',
    'Billing Issue': 'Medium',
    'Returns and Exchanges': 'Medium',
    'Service Outage': 'High',
    'Product Support': 'Low',
    'Sales Inquiry': 'Low',
    'Network Issue': 'High',
    'Software Bug': 'High',
    'Customer Service': 'Low',
    'Login Issue': 'Medium',
    'Warranty Claim': 'Medium',
    'Technical Guidance': 'Low',
    'Feature Request': 'Low',
    'Order Issue': 'Medium',
    'Hardware Failure': 'High',
    'Account Assistance': 'Medium',
    'Payment Processing': 'Medium'
}
```

```python
# Assign priority based on predicted label
df_test['priority'] = df_test['pred_label'].map(priority_mapping)

# Show sample
print(df_test.head())

# Save for next agent
df_test.to_csv("df_test_with_priority.csv", index=False)
print("Saved test dataset with priority scores as df_test_with_priority.csv")
```

```
                                                text  true_label  \
0  dear it services support team i am writing to ...          20
1  dear customer support i hope this email finds ...          14
2  dear tech online store customer support i hope...          20
3  dear it services our network has been down due...           9
4  transaction error message for adobe photoshop ...          11

          pred_label priority
0  Technical Support     High
1  Technical Support     High
2  Technical Support     High
3  Technical Support     High
4  Technical Support     High
Saved test dataset with priority scores as df_test_with_priority.csv
```

[redacted]

[redacted]

Agent 4: Response Suggestion Agent

```python
import pandas as pd

# Load dataset from previous agent
df_test = pd.read_csv("df_test_with_priority.csv")

# Define response templates based on category
response_templates = {
    'Technical Support': "Dear Customer, our technical team is reviewing your issue. We will get back to you shortly.",
    'IT Support': "Dear Customer, your IT support request has been logged. Our team will contact you soon.",
    'Urgent Issue': "Dear Customer, your urgent issue is being prioritized. Expect immediate assistance.",
    'Billing Issue': "Dear Customer, our billing team is reviewing your query and will respond shortly.",
    'Returns and Exchanges': "Dear Customer, your request for returns/exchanges is being processed.",
    'Service Outage': "Dear Customer, we are aware of the service outage and are working to resolve it.",
    'Product Support': "Dear Customer, our product support team will assist you with your query.",
    'Sales Inquiry': "Dear Customer, thank you for your inquiry. Our sales team will contact you soon.",
    'Network Issue': "Dear Customer, network issue reported. Our engineers are investigating.",
    'Software Bug': "Dear Customer, our development team is looking into the reported bug.",
    'Customer Service': "Dear Customer, our support team will respond to your request shortly.",
    'Login Issue': "Dear Customer, your login issue has been reported. Assistance will follow soon.",
    'Warranty Claim': "Dear Customer, your warranty claim is being reviewed.",
    'Technical Guidance': "Dear Customer, our team will provide technical guidance shortly.",
    'Feature Request': "Dear Customer, thank you for your suggestion. Our team will review it.",
    'Order Issue': "Dear Customer, your order issue has been logged. We will contact you soon.",
    'Hardware Failure': "Dear Customer, we are investigating the reported hardware failure.",
    'Account Assistance': "Dear Customer, our team will assist you with your account query.",
    'Payment Processing': "Dear Customer, your payment issue has been reported. Our team will contact you."
}

# Generate response based on predicted label
df_test['suggested_response'] = df_test['pred_label'].map(response_templates)

# Show sample responses
print(df_test[['text', 'pred_label', 'priority', 'suggested_response']].head())
```

```python
# Generate response based on predicted label
df_test['suggested_response'] = df_test['pred_label'].map(response_templates)

# Show sample responses
print(df_test[['text', 'pred_label', 'priority', 'suggested_response']].head())

# Save for next agent
df_test.to_csv("df_test_with_responses.csv", index=False)
print("Saved dataset with suggested responses as df_test_with_responses.csv")
```

```
                                              text         pred_label  \
0  dear it services support team i am writing to ...  Technical Support
1  dear customer support i hope this email finds ...  Technical Support
2  dear tech online store customer support i hope...  Technical Support
3  dear it services our network has been down due...  Technical Support
4  transaction error message for adobe photoshop ...  Technical Support

  priority                                suggested_response
0     High  Dear Customer, our technical team is reviewing...
1     High  Dear Customer, our technical team is reviewing...
2     High  Dear Customer, our technical team is reviewing...
3     High  Dear Customer, our technical team is reviewing...
4     High  Dear Customer, our technical team is reviewing...
Saved dataset with suggested responses as df_test_with_responses.csv
```

[redacted]

[redacted]

[redacted]

Agent 5: Feedback and Continuous Learning Agent

```python
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
import pickle

# Load dataset with suggested responses
df_feedback = pd.read_csv("df_test_with_responses.csv")

# Simulate feedback: user corrects label or priority
# For demonstration, let's assume a 'correct_label' column exists
# df_feedback['correct_label'] = user_feedback_labels  # Collected feedback

# If correct_label exists, update dataset
if 'correct_label' in df_feedback.columns:
    df_feedback['updated_label'] = df_feedback['correct_label']
else:
    df_feedback['updated_label'] = df_feedback['pred_label']

# Optionally update priority based on corrected label
priority_mapping = {
    'Urgent Issue': 'High',
    'Technical Support': 'High',
    'IT Support': 'Medium',
    'Billing Issue': 'Medium',
    'Returns and Exchanges': 'Medium',
    'Service Outage': 'High',
    'Product Support': 'Low',
    'Sales Inquiry': 'Low',
    'Network Issue': 'High',
    'Software Bug': 'High',
    'Customer Service': 'Low',
    'Login Issue': 'Medium',
    'Warranty Claim': 'Medium',
```

```python
df_feedback['updated_priority'] = df_feedback['updated_label'].map(priority_mapping)

# Save updated dataset
df_feedback.to_csv("df_feedback_updated.csv", index=False)
print("Saved updated dataset with feedback as df_feedback_updated.csv")

# retrain Random Forest with feedback
# Only do this if new labels exist
if 'correct_label' in df_feedback.columns:
    X_train = df_feedback['text']
    y_train = df_feedback['updated_label']
    tfidf = TfidfVectorizer(max_features=5000)
    X_train_tfidf = tfidf.fit_transform(X_train)
    rf = RandomForestClassifier(n_estimators=200, random_state=42)
    rf.fit(X_train_tfidf, y_train)

    # Save retrained model and vectorizer
    with open("rf_model_retrained.pkl", "wb") as f:
        pickle.dump(rf, f)
    with open("tfidf_retrained.pkl", "wb") as f:
        pickle.dump(tfidf, f)
    print("Retrained Random Forest model saved as rf_model_retrained.pkl")
```

```
Saved updated dataset with feedback as df_feedback_updated.csv
```

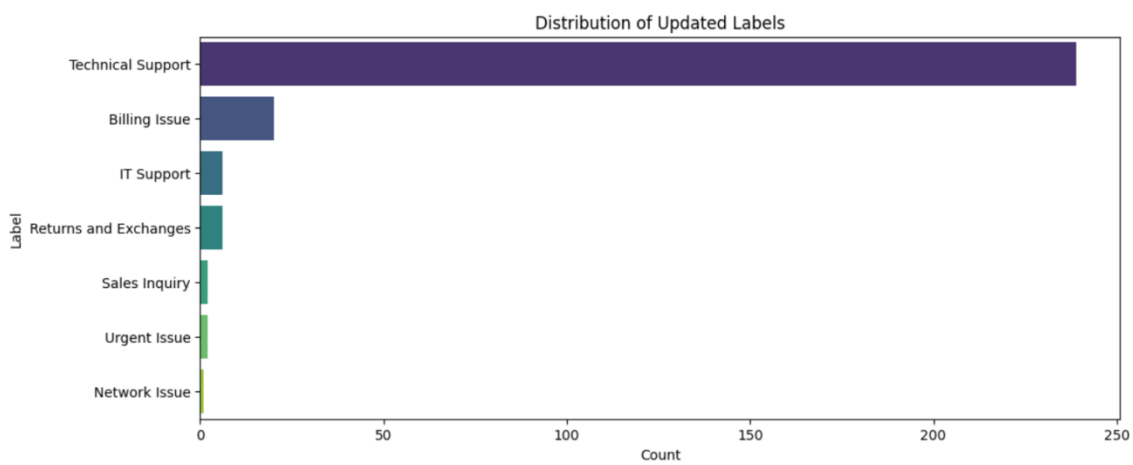
```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the updated dataset
df = pd.read_csv("df_feedback_updated.csv")

# Basic info
print("Dataset info:")
print(df.info())

# Display first few rows
```

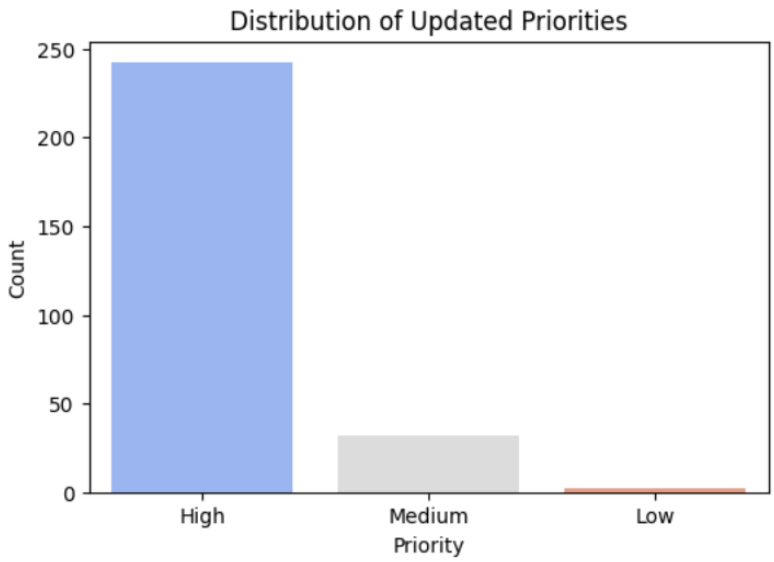
s Terminal

```
Saved updated dataset with feedback as df_feedback_updated.csv
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the updated dataset
df = pd.read_csv("df_feedback_updated.csv")

# Basic info
print("Dataset info:")
print(df.info())

# Display first few rows
print("\nSample rows:")
print(df.head())

# 1. Label distribution: predicted vs updated
plt.figure(figsize=(12,5))
sns.countplot(y='updated_label', data=df, order=df['updated_label'].value_counts().index, palette='viridis')
plt.title("Distribution of Updated Labels")
plt.xlabel("Count")
plt.ylabel("Label")
plt.show()

# 2. Priority distribution
plt.figure(figsize=(6,4))
sns.countplot(x='updated_priority', data=df, order=df['updated_priority'].value_counts().index, palette='coolwarm')
plt.title("Distribution of Updated Priorities")
plt.xlabel("Priority")
plt.ylabel("Count")
plt.show()

# 3. Compare predicted vs updated labels (discrepancies)
discrepancies = df[df['pred_label'] != df['updated_label']]
print(f"\nNumber of discrepancies: {len(discrepancies)}")
```

```
Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 276 entries, 0 to 275
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   text                276 non-null    object
 1   true_label          276 non-null    int64
 2   pred_label          276 non-null    object
 3   priority            276 non-null    object
 4   suggested_response  276 non-null    object
 5   updated_label       276 non-null    object
 6   updated_priority    276 non-null    object
dtypes: int64(1), object(6)
memory usage: 15.2+ KB
None

Sample rows:
                                                text  true_label  \
0  dear it services support team i am writing to ...          20
1  dear customer support i hope this email finds ...          14
2  dear tech online store customer support i hope...          20
3  dear it services our network has been down due...           9
4  transaction error message for adobe photoshop ...          11

          pred_label priority  \
0  Technical Support     High
1  Technical Support     High
2  Technical Support     High
3  Technical Support     High
4  Technical Support     High

                                 suggested_response      updated_label  \
0  Dear Customer, our technical team is reviewing...  Technical Support
1  Dear Customer, our technical team is reviewing...  Technical Support
2  Dear Customer, our technical team is reviewing...  Technical Support
3  Dear Customer, our technical team is reviewing...  Technical Support
4  Dear Customer, our technical team is reviewing...  Technical Support

  updated priority
```






```
Number of discrepancies: 0

Discrepancy sample:
Empty DataFrame
Columns: [text, pred_label, updated_label, priority, updated_priority]
Index: []
```

```
Number of discrepancies: 0

Discrepancy sample:
Empty DataFrame
Columns: [text, pred_label, updated_label, priority, updated_priority]
Index: []

Top 10 suggested responses:
suggested_response
Dear Customer, our technical team is reviewing your issue. We will get back to you shortly.    239
Dear Customer, our billing team is reviewing your query and will respond shortly.               20
Dear Customer, your IT support request has been logged. Our team will contact you soon.          6
Dear Customer, your request for returns/exchanges is being processed.                            6
Dear Customer, thank you for your inquiry. Our sales team will contact you soon.                 2
Dear Customer, your urgent issue is being prioritized. Expect immediate assistance.              2
Dear Customer, network issue reported. Our engineers are investigating.                          1
Name: count, dtype: int64
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]